

PYTHON OBJECT RELATIONSHIPS

객체 간의 관계: **Is-a** 와 **Has-a**

"객체지향 설계의 기초: 상속이냐, 조립이냐"

Chapter 4. Object Relationships

1. 객체는 태생적인 관계를 맺습니다

"메시지를 주고받는 동적 상호작용 이전에,
구조적으로 어떤 사이인지 정의하는 것이 관계입니다."



Is-a 관계

"~은 ~의 일종이다"
(혈연/계층 관계)

VS



Has-a 관계

"~은 ~을 가진다"
(소유/구성 관계)

2-1. Is-a 관계: 상속 (Inheritance)

"A is a B" (A는 B의 일종이다)



Student **is a** Person



Dog **is a** Animal

상속의 의미

- 부모(A)는 일반적(**General**)인 개념
- 자식(B)는 구체적(**Specific**)인 개념
- 자식은 부모의 속성과 행동을 그대로 물려받음
- 목적: 코드 재사용 및 계층 구조 표현

2-2. 파이썬 코드로 보는 Is-a

괄호 안에 부모 클래스 이름을 넣어 상속받습니다.

```
# 1. 부모 클래스 (일반적인 사람)
class Person:
    def eat(self):
        print("밥을 먹습니다.")

# 2. 자식 클래스 (구체적인 학생)
# Student is a Person (학생은 사람이다)
class Student(Person):
    def study(self):
        print("공부를 합니다.")

# 3. 사용
s = Student()
s.eat()    # 부모에게 물려받은 기능 사용 가능!
s.study()  # 자신의 기능
```

2-3. 🚨 잘못된 상속의 예 (Bad Inheritance)

"말이 된다고 무조건 상속하면 괴물 객체가 탄생합니다."

Case 1: 경찰 is a 총?



경찰이 총을 쓰고 싶다고 상속받으면?

- 👉 경찰.발사()가 가능해짐.
- 👉 경찰은 총이 아닙니다. 총을 가져야(Has-a)합니다.

Case 2: 펭귄 is a 새?



생물학적으로는 맞지만 프로그래밍에선?

- 👉 부모(새)는 fly() 기능이 있음.
- 👉 펭귄은 못 낚. 상속받으면 에러 발생!
- 👉 행위(Behavior)가 일치해야 상속 가능.

"상속은 비슷해 보여서 쓰는 게 아니라,
대체 가능해야 쓴다." (LSP 원칙)

3-1. Has-a 관계: 포함 (Composition)

"A has a B" (A는 B를 가지고 있다 / 구성요소다)



Car **has a** Engine



Police **has a** Gun

포함의 의미

- 전체(Whole)와 부품(Part)의 관계
- 상속보다 **유연한 결합** (부품 교체 가능)
- 내 기능을 내가 직접 하지 않고, **부품에게 시킴 (위임)**

3-2. 파이썬 코드로 보는 Has-a

생성자(__init__)에서 객체를 생성하여 멤버 변수로 가집니다.

1. 부품 클래스

```
class Engine:
    def start(self):
        print("부릉! 엔진 가동")
```

2. 전체 클래스

```
class Car:
    def __init__(self):
        # Car has a Engine (엔진을 소유함)
        self.engine = Engine()

    def drive(self):
        # [위임] 내가 직접 돌리는 게 아니라 엔진에게 시킴
        self.engine.start()
        print("출발합니다!")
```

3. 사용

```
my_car = Car()
my_car.drive()
```

3-3. Has-a의 두 가지 얼굴

"생명주기(Lifecycle)와 결합도에 따라 나뉩니다."

합성 (Composition)

강한 결합 (Strong)



사람 & 심장

"사람이 죽으면 심장도 멈춘다."
(생명주기 동일)"

집합 (Aggregation)

약한 결합 (Weak)



학교 & 학생

"학교가 사라져도 학생은 살아있다."
(생명주기 독립)"

4. 한눈에 보는 비교 (Is-a vs Has-a)

구분	Is-a (상속)	Has-a (포함)
의미	A는 B의 일종이다 (Kind of)	A는 B를 부품으로 가진다 (Part of)
구현 방식	<code>class A(B):</code> (상속 문법 사용)	<code>self.b = B()</code> (멤버 변수로 객체 소유)
결합도	강함 (Tight) 부모 변경 시 자식에게 치명적 영향	유연함 (Loose) 부품 교체가 쉬움
관계	수직적 (계층 구조)	수평적 (조립 구조)

5. 베테랑의 설계 원칙

"상속(Is-a)보다는 합성(Has-a)을 우선하라"

Favor Composition over Inheritance

상속의 문제점

- 캡슐화 위반: 자식이 부모 내부를 다 알아야 함.
- 유연성 부족: 실행 중에 부모를 바꿀 수 없음 (정적).
- 클래스 폭발: 기능 조합을 위해 수많은 클래스가 생김.

합성의 장점

- 블랙박스 재사용: 내부를 몰라도 인터페이스만 알면 됨.
- 런타임 교체: 실행 중에 부품(객체)을 갈아끼울 수 있음.
- 단순성: 클래스 수를 줄이고 관계를 단순화.

6. 결론



객체지향의 핵심은
~~'거대한 상속 계층 만들기'~~가 아니라
'적절한 책임을 나누고 조립하기'입니다.

"상속은 정말 확실한 'Is-a'일 때만 사용하세요."

7. 실전 퀴즈: Is-a일까? Has-a일까?

"한번 맞춰보세요! (정답은 다음 페이지)"



1. 노트북 vs 컴퓨터

노트북 ↔ 컴퓨터



2. 회원 vs 비밀번호

회원 ↔ 비밀번호



3. 게임캐릭터 vs 무기

캐릭터 ↔ 무기



4. List vs ArrayList

ArrayList ↔ List



5. 스마트폰 vs 카메라

스마트폰 ↔ 카메라



6. 정사각형 vs 직사각형

정사각형 ↔ 직사각형

정답 확인

(카드로 한눈에 보기)

1. 노트북 vs 컴퓨터

Is-a (상속)

노트북은 컴퓨터의 일종이다.

2. 회원 vs 비밀번호

Has-a (포함)

회원은 비밀번호를 가진다.

3. 게임캐릭터 vs 무기

Has-a (포함)

캐릭터는 무기를 소유한다.

4. List vs ArrayList

Is-a (상속)

ArrayList는 List의 일종이다.

5. 스마트폰 vs 카메라

Has-a (포함)

폰은 카메라 모듈을 가진다.

6. 정사각형 vs 직사각형

주의! (Has-a)

LSP 위반 방지를 위해 포함 추천.